

STUDENT LABORATORY MANUAL

Module 3

Interactive Computer Graphics with OpenGL and GLUT

5 Guided Keyboard Programs • 3 Machine Problems
CS0045 – Computer Graphics and Visual Computing

Module Focus Keyboard callbacks, bitmap text, OpenGL primitives, color state, movement, boundary checking, collision detection, HUDs, scoring, and timer-driven animation.

Toolchain: C++ • OpenGL • GLUT / FreeGLUT

Name	SERRANO, LIAM BRENT C.
Section	TN35
Github	https://github.com/TGOLiam/CS0045-M3-EXERCISE-2

How to Use This Manual

This laboratory manual is adapted from the Module 3 workflow and uses the supplied m3ex1.cpp through m3ex5.cpp programs as the five guided examples. The original Module 3 manual emphasizes complete, runnable examples followed by independent exercises, compiled in the shared OpenGL workspace. The same progression is retained here, but the guided work is focused on keyboard-driven interaction and small game-like graphics.

Laboratory Workflow

- Read the concept and expected output before examining the code.
- Type or copy the program into your Module 3 source folder and compile it.
- Run the program and test every keyboard command.
- Trace how the keyboard callback changes global state and how glutPostRedisplay() refreshes the scene.
- Complete the Try It tasks, then proceed to the Machine Problems without copying a finished solution.

Common Header Block

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
```

Important The source programs use glutBitmapCharacter() in a helper function for portable bitmap text rendering. The Module 3 reference manual likewise uses a character-by-character helper for bitmap text and demonstrates keyboard callbacks, redraw requests, HUD text, idle animation, and self-rescheduling timers.

Learning Outcomes

- Render 2D text and primitive shapes in an OpenGL window.
- Use glutKeyboardFunc() to map keys to program actions.
- Maintain program state with global variables and update the display interactively.
- Apply movement constraints and simple axis-aligned collision checks.
- Build a basic heads-up display for score, status, lives, and instructions.
- Extend event-driven programs into timer-driven interactive simulations.

Program 1: RGB Background Controller

Concept: Keyboard-driven color state and dynamic bitmap-text contrast.

Expected Output: A 700 x 500 window whose background changes immediately when the user presses the color keys. The instruction text remains visible by using a contrasting color.

Learning Objectives

- Use RGB floating-point values with `glClearColor()`.
- Handle upper- and lowercase keyboard input with a switch statement.
- Request a redraw after changing state.
- Render an on-screen instruction panel with bitmap text.

Program Walkthrough

State variables: red, green, and blue store the current background color.

Rendering: `display()` clears the window using the current RGB state and draws instructions in the inverse color for contrast.

Interaction: `keyboard()` maps R, G, B, W, and K to preset RGB combinations.

Redraw: `glutPostRedisplay()` asks GLUT to call `display()` again after a key changes the state.

Keyboard Controls

Key	Action
R	Red background
G	Green background
B	Blue background
W	White background
K	Black background
ESC	Exit

Source Code

```
#define GL_SILENCE_DEPRECATION

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif

#include <iostream>
#include <cstdlib>
```

```

#include <string>

using namespace std;

float red = 0.0f;
float green = 0.0f;
float blue = 0.0f;

// Function to draw bitmap text on the screen
void drawText(float x, float y, const string& text)
{
    glRasterPos2f(x, y);
    for (char c : text)
    {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, c);
    }
}

void display()
{
    glClearColor(red, green, blue, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Set text color to contrast dynamically with the background color
    glColor3f(1.0f - red, 1.0f - green, 1.0f - blue);

    // Display instructions on the screen
    drawText(-0.8f, 0.6f, "=== RGB BACKGROUND CONTROLLER ===");
    drawText(-0.8f, 0.4f, "[R] - Change to RED");
    drawText(-0.8f, 0.2f, "[G] - Change to GREEN");
    drawText(-0.8f, 0.0f, "[B] - Change to BLUE");
    drawText(-0.8f, -0.2f, "[W] - Change to WHITE");
    drawText(-0.8f, -0.4f, "[K] - Change to BLACK");
    drawText(-0.8f, -0.6f, "[ESC] - Exit Program");

    glFlush();
}

void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'r':
        case 'R':
            red = 1.0f;
            green = 0.0f;
            blue = 0.0f;
            cout << "Background: RED" << endl;
            break;

        case 'g':
        case 'G':
            red = 0.0f;
            green = 1.0f;
            blue = 0.0f;
            cout << "Background: GREEN" << endl;
            break;

        case 'b':
        case 'B':
            red = 0.0f;
            green = 0.0f;

```

```

        blue = 1.0f;
        cout << "Background: BLUE" << endl;
        break;

    case 'w':
    case 'W':
        red = 1.0f;
        green = 1.0f;
        blue = 1.0f;
        cout << "Background: WHITE" << endl;
        break;

    case 'k':
    case 'K':
        red = 0.0f;
        green = 0.0f;
        blue = 0.0f;
        cout << "Background: BLACK" << endl;
        break;

    case 27:
        exit(0);
}

glutPostRedisplay();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);

    glutInitWindowSize(700, 500);
    glutInitWindowPosition(200, 100);

    glutCreateWindow("Program 1 - RGB Background Controller with Text");

    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);

    glutMainLoop();

    return 0;
}

```

Try It

- Compile and run the program before changing any code.
- Use every listed keyboard command and observe both the graphics window and console output.
- Change one numeric value (color, movement step, object size, or boundary) and predict the result before running again.

Program 2: Keyboard-Controlled Square

Concept: Two-axis keyboard movement with boundary clamping.

Expected Output: A cyan square moves in four directions over a dark background while remaining within the defined coordinate limits.

Learning Objectives

- Represent a movable object using X/Y position variables.
- Update coordinates from W/A/S/D keyboard input.
- Clamp movement so the object remains in the visible area.
- Display controls while rendering a primitive with GL_QUADS.

Program Walkthrough

Position state: squareX and squareY store the center of the square.

Movement step: Each valid movement key changes one coordinate by 0.05.

Boundary checking: After movement, each coordinate is limited to approximately -0.9 to 0.9.

Feedback: The current position is printed to the console and the graphics window is redrawn.

Keyboard Controls

Key	Action
W	Move up
S	Move down
A	Move left
D	Move right
R	Reset to center
ESC	Exit

Source Code

```
#define GL_SILENCE_DEPRECATION

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif

#include <iostream>
#include <cstdlib>
#include <string>
```

```

using namespace std;

float squareX = 0.0f;
float squareY = 0.0f;

// Function to draw bitmap text on the screen
void drawText(float x, float y, const string& text)
{
    glRasterPos2f(x, y);
    for (char c : text)
    {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, c);
    }
}

void display()
{
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Set text color to white for contrast against the dark background
    glColor3f(1.0f, 1.0f, 1.0f);

    // Display movement instructions on the screen
    drawText(-0.9f, 0.82f, "=== SQUARE CONTROLLER ===");
    drawText(-0.9f, 0.72f, "[W] / [S] - Move Up / Down");
    drawText(-0.9f, 0.62f, "[A] / [D] - Move Left / Right");
    drawText(-0.9f, 0.52f, "[R]      - Reset Position");
    drawText(-0.9f, 0.42f, "[ESC]   - Exit Program");

    // Draw the movable square
    glColor3f(0.0f, 0.7f, 1.0f);
    glBegin(GL_QUADS);
        glVertex2f(squareX - 0.1f, squareY - 0.1f);
        glVertex2f(squareX + 0.1f, squareY - 0.1f);
        glVertex2f(squareX + 0.1f, squareY + 0.1f);
        glVertex2f(squareX - 0.1f, squareY + 0.1f);
    glEnd();

    glFlush();
}

void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'w':
        case 'W':
            squareY += 0.05f;
            break;

        case 's':
        case 'S':
            squareY -= 0.05f;
            break;

        case 'a':
        case 'A':
            squareX -= 0.05f;
            break;
    }
}

```

```

    case 'd':
    case 'D':
        squareX += 0.05f;
        break;

    case 'r':
    case 'R':
        squareX = 0.0f;
        squareY = 0.0f;
        break;

    case 27:
        exit(0);
}

// Prevent square from leaving screen
if (squareX > 0.9f)
    squareX = 0.9f;

if (squareX < -0.9f)
    squareX = -0.9f;

if (squareY > 0.9f)
    squareY = 0.9f;

if (squareY < -0.9f)
    squareY = -0.9f;

cout << "Position: "
    << squareX << ", "
    << squareY << endl;

glutPostRedisplay();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);

    glutInitWindowSize(700, 700);

    glutCreateWindow("Program 2 - Keyboard Controlled Square with Text");

    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);

    glutMainLoop();

    return 0;
}

```

Try It

- Compile and run the program before changing any code.
- Use every listed keyboard command and observe both the graphics window and console output.
- Change one numeric value (color, movement step, object size, or boundary) and predict the result before running again.

Program 3: Interactive Shape & Color Selector

Concept: Multiple drawing functions controlled by shape and color state.

Expected Output: The user switches among three centered shapes and changes the selected shape to red, green, or blue without restarting the program.

Learning Objectives

- Create reusable functions for triangle, square, and diamond primitives.
- Use a state variable to select which shape is rendered.
- Use RGB variables to change the active shape color.
- Combine numeric and alphabetic keyboard commands in one callback.

Program Walkthrough

Shape state: shape identifies the active primitive: 1 triangle, 2 square, 3 diamond.

Color state: red, green, and blue hold the selected shape color.

Modular drawing: drawTriangle(), drawSquare(), and drawDiamond() isolate geometry from interaction logic.

Selection: The keyboard callback updates state and display() decides which drawing function to call.

Keyboard Controls

Key	Action
1	Triangle
2	Square
3	Diamond
R	Red
G	Green
B	Blue
ESC	Exit

Source Code

```
#define GL_SILENCE_DEPRECATION

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
```

```

#include <iostream>
#include <cstdlib>
#include <string>

using namespace std;

int shape = 1;

float red = 0.0f;
float green = 0.7f;
float blue = 1.0f;

// Function to draw bitmap text on the screen
void drawText(float x, float y, const string& text)
{
    glRasterPos2f(x, y);
    for (char c : text)
    {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, c);
    }
}

void drawTriangle()
{
    glBegin(GL_TRIANGLES);
    glVertex2f(0.0f, 0.4f);
    glVertex2f(-0.4f, -0.3f);
    glVertex2f(0.4f, -0.3f);
    glEnd();
}

void drawSquare()
{
    glBegin(GL_QUADS);
    glVertex2f(-0.3f, -0.3f);
    glVertex2f(0.3f, -0.3f);
    glVertex2f(0.3f, 0.3f);
    glVertex2f(-0.3f, 0.3f);
    glEnd();
}

void drawDiamond()
{
    glBegin(GL_POLYGON);
    glVertex2f(0.0f, 0.4f);
    glVertex2f(0.4f, 0.0f);
    glVertex2f(0.0f, -0.4f);
    glVertex2f(-0.4f, 0.0f);
    glEnd();
}

void display()
{
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Set text color to white for contrast against dark background
    glColor3f(1.0f, 1.0f, 1.0f);

    // Display instruction text panel
    drawText(-0.9f, 0.82f, "=== SHAPE & COLOR SELECTOR ===");
}

```

```

drawText(-0.9f, 0.72f, "[1] - Triangle | [R] - Red Color");
drawText(-0.9f, 0.62f, "[2] - Square | [G] - Green Color");
drawText(-0.9f, 0.52f, "[3] - Diamond | [B] - Blue Color");
drawText(-0.9f, 0.42f, "[ESC] - Exit Program");

// Set shape color based on user selection
glColor3f(red, green, blue);

if (shape == 1)
    drawTriangle();
else if (shape == 2)
    drawSquare();
else if (shape == 3)
    drawDiamond();

glFlush();
}

void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case '1':
            shape = 1;
            cout << "Triangle selected" << endl;
            break;

        case '2':
            shape = 2;
            cout << "Square selected" << endl;
            break;

        case '3':
            shape = 3;
            cout << "Diamond selected" << endl;
            break;

        case 'r':
        case 'R':
            red = 1.0f;
            green = 0.0f;
            blue = 0.0f;
            cout << "Color: RED" << endl;
            break;

        case 'g':
        case 'G':
            red = 0.0f;
            green = 1.0f;
            blue = 0.0f;
            cout << "Color: GREEN" << endl;
            break;

        case 'b':
        case 'B':
            red = 0.0f;
            green = 0.0f;
            blue = 1.0f;
            cout << "Color: BLUE" << endl;
            break;

        case 27:

```

```
        exit(0);
    }

    glutPostRedisplay();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);

    glutInitWindowSize(700, 600);

    glutCreateWindow("Program 3 - Interactive Shape Selector with Text");

    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);

    glutMainLoop();

    return 0;
}
```

Try It

- Compile and run the program before changing any code.
- Use every listed keyboard command and observe both the graphics window and console output.
- Change one numeric value (color, movement step, object size, or boundary) and predict the result before running again.

Program 4: Keyboard Car Controller

Concept: Composite-object drawing, horizontal movement, color changes, and bounds.

Expected Output: A simple car built from multiple primitives moves left and right and changes color while remaining inside the visible window.

Learning Objectives

- Construct a recognizable object from several OpenGL primitives.
- Move a composite object by applying the same X offset to its vertices.
- Change the object color from keyboard input.
- Clamp a wide object using limits appropriate to its dimensions.

Program Walkthrough

Composite object: drawCar() builds the body and roof from filled primitives and the wheels from GL_POINTS.

Shared offset: carX is added to every horizontal vertex so the whole car moves together.

Color control: carR, carG, and carB determine the body/roof color.

Boundary: carX is limited to -0.55 through 0.55 so the wide car remains visible.

Keyboard Controls

Key	Action
A	Move left
D	Move right
H	Return to center
R	Red car
G	Green car
B	Blue car
ESC	Exit

Source Code

```
#define GL_SILENCE_DEPRECATION

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
```

```

#include <iostream>
#include <cstdlib>
#include <string>

using namespace std;

float carX = 0.0f;

float carR = 0.0f;
float carG = 0.5f;
float carB = 1.0f;

// Function to draw bitmap text on the screen
void drawText(float x, float y, const string& text)
{
    glRasterPos2f(x, y);
    for (char c : text)
    {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, c);
    }
}

void drawCar()
{
    // Car body
    glColor3f(carR, carG, carB);
    glBegin(GL_QUADS);
        glVertex2f(carX - 0.4f, -0.2f);
        glVertex2f(carX + 0.4f, -0.2f);
        glVertex2f(carX + 0.4f, 0.1f);
        glVertex2f(carX - 0.4f, 0.1f);
    glEnd();

    // Car roof
    glBegin(GL_POLYGON);
        glVertex2f(carX - 0.25f, 0.1f);
        glVertex2f(carX - 0.1f, 0.35f);
        glVertex2f(carX + 0.2f, 0.35f);
        glVertex2f(carX + 0.3f, 0.1f);
    glEnd();

    // Wheels
    glColor3f(0.0f, 0.0f, 0.0f);
    glPointSize(35.0f);
    glBegin(GL_POINTS);
        glVertex2f(carX - 0.25f, -0.25f);
        glVertex2f(carX + 0.25f, -0.25f);
    glEnd();
}

void display()
{
    glClearColor(0.8f, 0.9f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Set text color to dark gray/black for contrast against the light blue background
    glColor3f(0.1f, 0.1f, 0.1f);

    // Display instructions on the screen
    drawText(-0.9f, 0.82f, "=== CAR CONTROLLER INSTRUCTIONS ===");
    drawText(-0.9f, 0.72f, "[A] / [D] - Move Car Left / Right");
}

```

```

drawText(-0.9f, 0.62f, "[H]    - Home / Center Car");
drawText(-0.9f, 0.52f, "[R] / [G] / [B] - Change Car Color (Red/Green/Blue)");
drawText(-0.9f, 0.42f, "[ESC]  - Exit Program");

// Draw the interactive car
drawCar();

glFlush();
}

void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'a':
        case 'A':
            carX -= 0.05f;
            break;

        case 'd':
        case 'D':
            carX += 0.05f;
            break;

        case 'h':
        case 'H':
            carX = 0.0f;
            break;

        case 'r':
        case 'R':
            carR = 1.0f;
            carG = 0.0f;
            carB = 0.0f;
            break;

        case 'g':
        case 'G':
            carR = 0.0f;
            carG = 1.0f;
            carB = 0.0f;
            break;

        case 'b':
        case 'B':
            carR = 0.0f;
            carG = 0.0f;
            carB = 1.0f;
            break;

        case 27:
            exit(0);
    }

    if (carX > 0.55f)
        carX = 0.55f;

    if (carX < -0.55f)
        carX = -0.55f;

    glutPostRedisplay();
}

```

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);

    glutInitWindowSize(900, 600);

    glutCreateWindow("Program 4 - Keyboard Car Controller with Text");

    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);

    glutMainLoop();

    return 0;
}
```

Try It

- Compile and run the program before changing any code.
- Use every listed keyboard command and observe both the graphics window and console output.
- Change one numeric value (color, movement step, object size, or boundary) and predict the result before running again.

Program 5: Collect the Target

Concept: Player movement, HUD rendering, collision detection, score tracking, and restart state.

Expected Output: A blue player square moves around the play area collecting a yellow target. Each collision increases the score and moves the target to another location.

Learning Objectives

- Combine movement, collision detection, scoring, and HUD text in one interactive program.
- Use a helper function to render dynamic score text.
- Detect overlap using X/Y distance thresholds.
- Reset state cleanly and reposition targets after collection.

Program Walkthrough

Player and target: Separate X/Y variables represent the movable player and collectible target.

HUD: `snprintf()` converts the integer score into display text, rendered with `drawBitmapString()`.

Collision: `collision()` computes absolute X/Y distances and reports a hit when both are below 0.15.

Target sequence: `moveTarget()` uses score % 6 to cycle through six predefined locations.

Restart: R restores player position, score, and target coordinates.

Keyboard Controls

Key	Action
W/A/S/D	Move player
R	Restart game
ESC	Exit

Source Code

```
#define GL_SILENCE_DEPRECATED

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif

#include <iostream>
#include <cstdio>
#include <cstdlib>

using namespace std;

float playerX = 0.0f;
float playerY = 0.0f;

float targetX = 0.6f;
```

```

float targetY = 0.5f;

int score = 0;

void drawBitmapString(void* font, const char* str)
{
    for (const char* c = str; *c != '\0'; c++)
    {
        glutBitmapCharacter(font, *c);
    }
}

void drawPlayer()
{
    glColor3f(0.0f, 0.6f, 1.0f);

    glBegin(GL_QUADS);
    glVertex2f(playerX - 0.08f, playerY - 0.08f);
    glVertex2f(playerX + 0.08f, playerY - 0.08f);
    glVertex2f(playerX + 0.08f, playerY + 0.08f);
    glVertex2f(playerX - 0.08f, playerY + 0.08f);
    glEnd();
}

void drawTarget()
{
    glColor3f(1.0f, 0.8f, 0.0f);

    glBegin(GL_QUADS);
    glVertex2f(targetX - 0.07f, targetY - 0.07f);
    glVertex2f(targetX + 0.07f, targetY - 0.07f);
    glVertex2f(targetX + 0.07f, targetY + 0.07f);
    glVertex2f(targetX - 0.07f, targetY + 0.07f);
    glEnd();
}

void drawHUD()
{
    char buffer[50];

    // Draw Score
    snprintf(buffer, sizeof(buffer), "Score: %d", score);
    glColor3f(1.0f, 1.0f, 1.0f);
    glRasterPos2f(-0.9f, 0.9f);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, buffer);

    // Draw On-Screen Instructions Panel
    glColor3f(0.8f, 0.8f, 0.8f); // Slightly softer white for instructions
    glRasterPos2f(-0.9f, 0.76f);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "=== CONTROLS ===");
    glRasterPos2f(-0.9f, 0.66f);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "[W][A][S][D] - Move Player");
    glRasterPos2f(-0.9f, 0.56f);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "[R]      - Restart Game");
    glRasterPos2f(-0.9f, 0.46f);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "[ESC]    - Exit Program");
}

bool collision()
{
    float distanceX = playerX - targetX;
    float distanceY = playerY - targetY;

```

```

    if (distanceX < 0)
        distanceX = -distanceX;

    if (distanceY < 0)
        distanceY = -distanceY;

    return distanceX < 0.15f &&
           distanceY < 0.15f;
}

void moveTarget()
{
    int position = score % 6;

    switch (position)
    {
        case 0:
            targetX = 0.6f;
            targetY = 0.5f;
            break;

        case 1:
            targetX = -0.6f;
            targetY = 0.5f;
            break;

        case 2:
            targetX = 0.6f;
            targetY = -0.5f;
            break;

        case 3:
            targetX = -0.6f;
            targetY = -0.5f;
            break;

        case 4:
            targetX = 0.0f;
            targetY = 0.7f;
            break;

        case 5:
            targetX = 0.0f;
            targetY = -0.7f;
            break;
    }
}

void display()
{
    glClearColor(0.08f, 0.08f, 0.12f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    drawPlayer();
    drawTarget();
    drawHUD();

    glFlush();
}

void keyboard(unsigned char key, int x, int y)

```

```

{
    switch (key)
    {
        case 'w':
        case 'W':
            playerY += 0.08f;
            break;

        case 's':
        case 'S':
            playerY -= 0.08f;
            break;

        case 'a':
        case 'A':
            playerX -= 0.08f;
            break;

        case 'd':
        case 'D':
            playerX += 0.08f;
            break;

        case 'r':
        case 'R':
            playerX = 0.0f;
            playerY = 0.0f;
            score = 0;
            targetX = 0.6f;
            targetY = 0.5f;
            cout << "Game restarted." << endl;
            break;

        case 27:
            exit(0);
    }

    // Boundary checking
    if (playerX > 0.9f)
        playerX = 0.9f;

    if (playerX < -0.9f)
        playerX = -0.9f;

    if (playerY > 0.85f)
        playerY = 0.85f;

    if (playerY < -0.85f)
        playerY = -0.85f;

    // Collision detection
    if (collision())
    {
        score++;
        cout << "Target collected! Score: " << score << endl;
        moveTarget();
    }

    glutPostRedisplay();
}

int main(int argc, char** argv)

```

```
{  
    glutInit(&argc, argv);  
    glutInitWindowSize(800, 700);  
    glutInitWindowPosition(200, 100);  
  
    glutCreateWindow("Program 5 - Collect the Target with Text Instructions");  
  
    glutDisplayFunc(display);  
    glutKeyboardFunc(keyboard);  
  
    glutMainLoop();  
  
    return 0;  
}
```

Try It

- Compile and run the program before changing any code.
- Use every listed keyboard command and observe both the graphics window and console output.
- Change one numeric value (color, movement step, object size, or boundary) and predict the result before running again.

Exercise Section — Machine Problems

Complete the following programs independently. They are intentionally designed as extensions of the five guided examples. Reuse concepts, not entire finished solutions. Your submission should compile, run, and provide clear on-screen instructions.

Submission Standard For each machine problem, submit the .cpp source file, one screenshot showing the running program, and a short reflection answering the guide questions. Use meaningful function and variable names and include comments for major logic blocks.

SOURCE CODE LINK
https://github.com/TGOLiam/CS0045-M3-EXERCISE

Machine Problem 1: “The Cosmic Dodger”

Difficulty	Intermediate
Concept	Builds upon the player movement and target-collection mechanics from Program 5, while adding automatic hazards, lives, and a game-over state.

Objective

Create a space-themed survival game where the player controls a custom-drawn spaceship that can move freely within bounded coordinates.

Key Requirements

- **Player Ship:** Draw a recognizable spaceship from at least two OpenGL primitives. The ship must move using keyboard controls and stay inside the play area.
- **Hazard Spawns:** Create at least two asteroid obstacles, rendered as polygons or squares, that move automatically downward from the top at different speeds.
- **Lives & HUD:** Start with 3 lives. Display both Score and Lives clearly using `drawBitmapString()` or the same character-by-character bitmap-text helper used in the guided programs.
- **Collision Penalty:** If an asteroid hits the player, subtract 1 life, print a console notification, and safely reset that asteroid to the top.
- **Game Over State:** When lives reach 0, stop gameplay updates, clear or simplify the play area, and display: GAME OVER - Press R to Restart.
- **Restart:** R must restore lives, score, player position, asteroid positions, and the running state.

Minimum Acceptance Checklist

- ☐ W/A/S/D or arrow-style controls move the spaceship.
- ☐ At least two hazards must have visibly different falling speeds.
- ☐ The HUD must remain readable during play.
- ☐ Collision must not subtract multiple lives continuously from one overlap.
- ☐ The game must be restartable without closing the window.

Recommended Program Structure

- `drawShip()`
- `drawAsteroid(...)`
- `drawHUD()`
- `checkCollision(...)`
- `updateGame(int value)`
- `resetGame()`
- `keyboard(...)`
- `display()`

Reflection Questions

1. How did you prevent repeated life loss from a single collision?

Repeated life loss is prevented by resetting the collided asteroid and immediately returning after one collision check.

2. Which variables define the complete game state that must be reset?

The complete reset state includes lives, score, running status, thrust/direction/velocity, ship position, asteroid spawn timing, and the asteroid list.

3. How does your animation update the asteroid positions over time?

A timer callback runs about every 16 ms, uses elapsed time as delta time, and moves each asteroid downward according to its velocity.

Testing Record

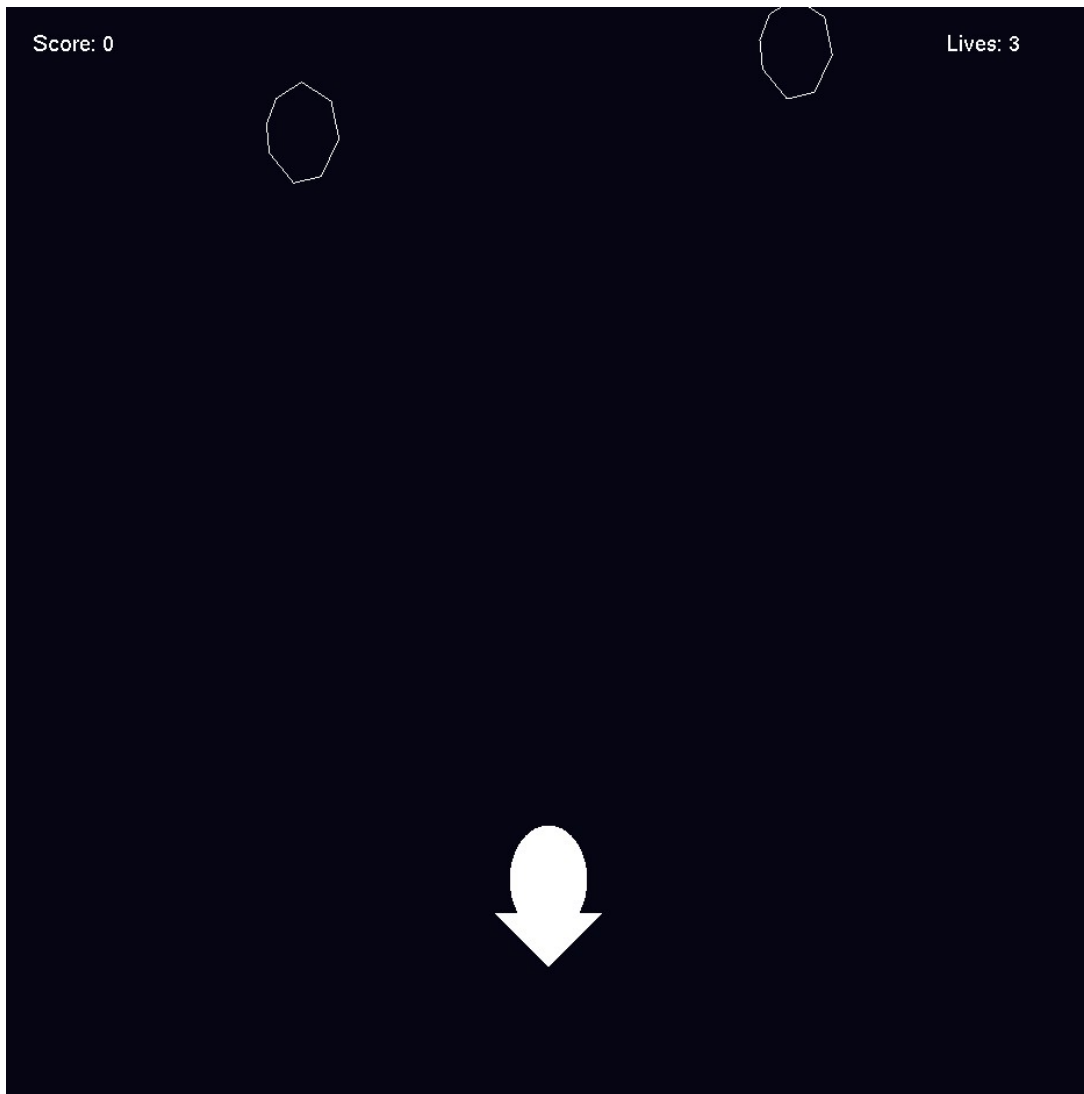
Input/Action

Launch the program and observe the initial game state.

Expected Result

The ship, score, lives, and falling hazards appear in the game window.

Actual Result

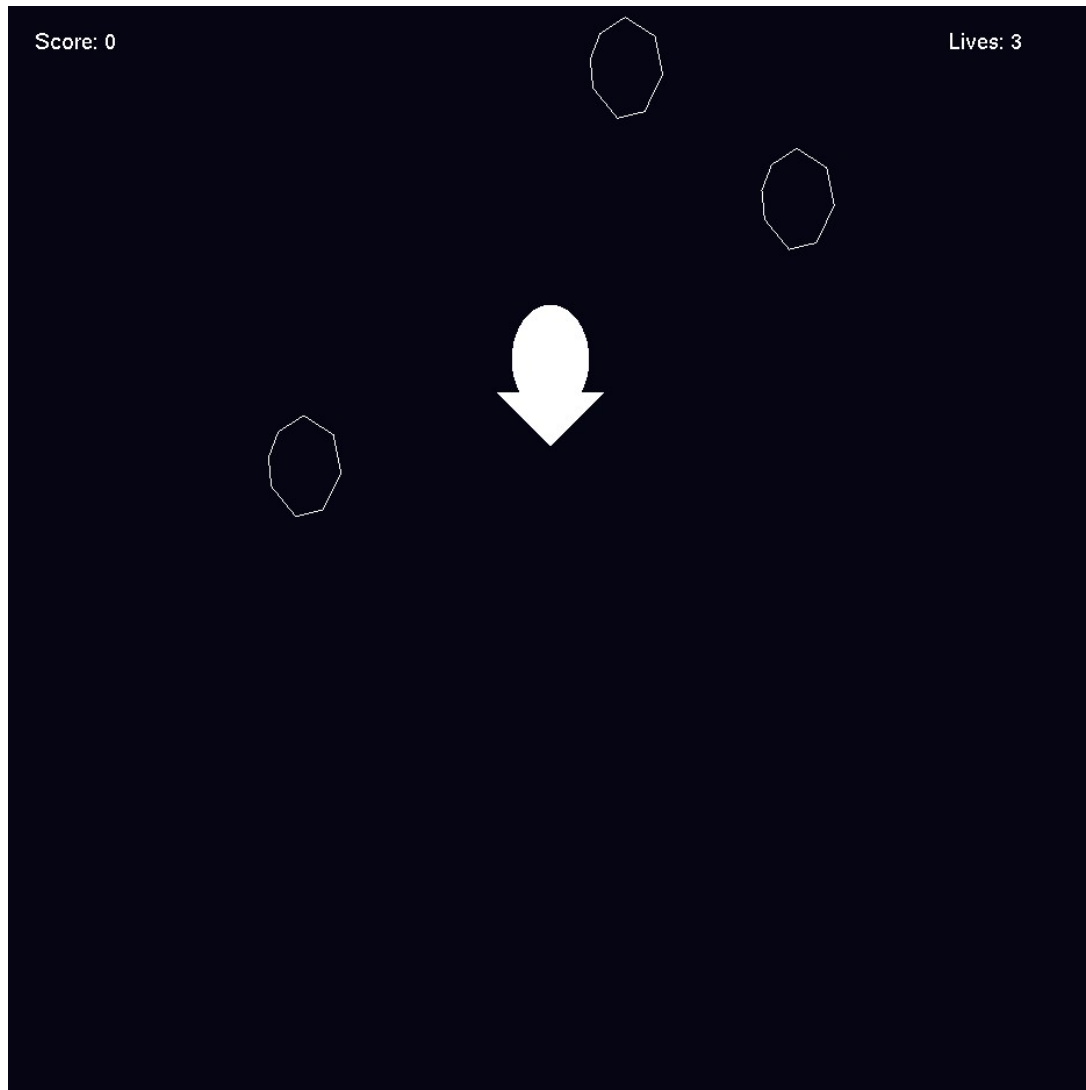


Input/Action

Hold W briefly to move the ship upward.

Expected Result

The ship moves upward while remaining inside the play area.

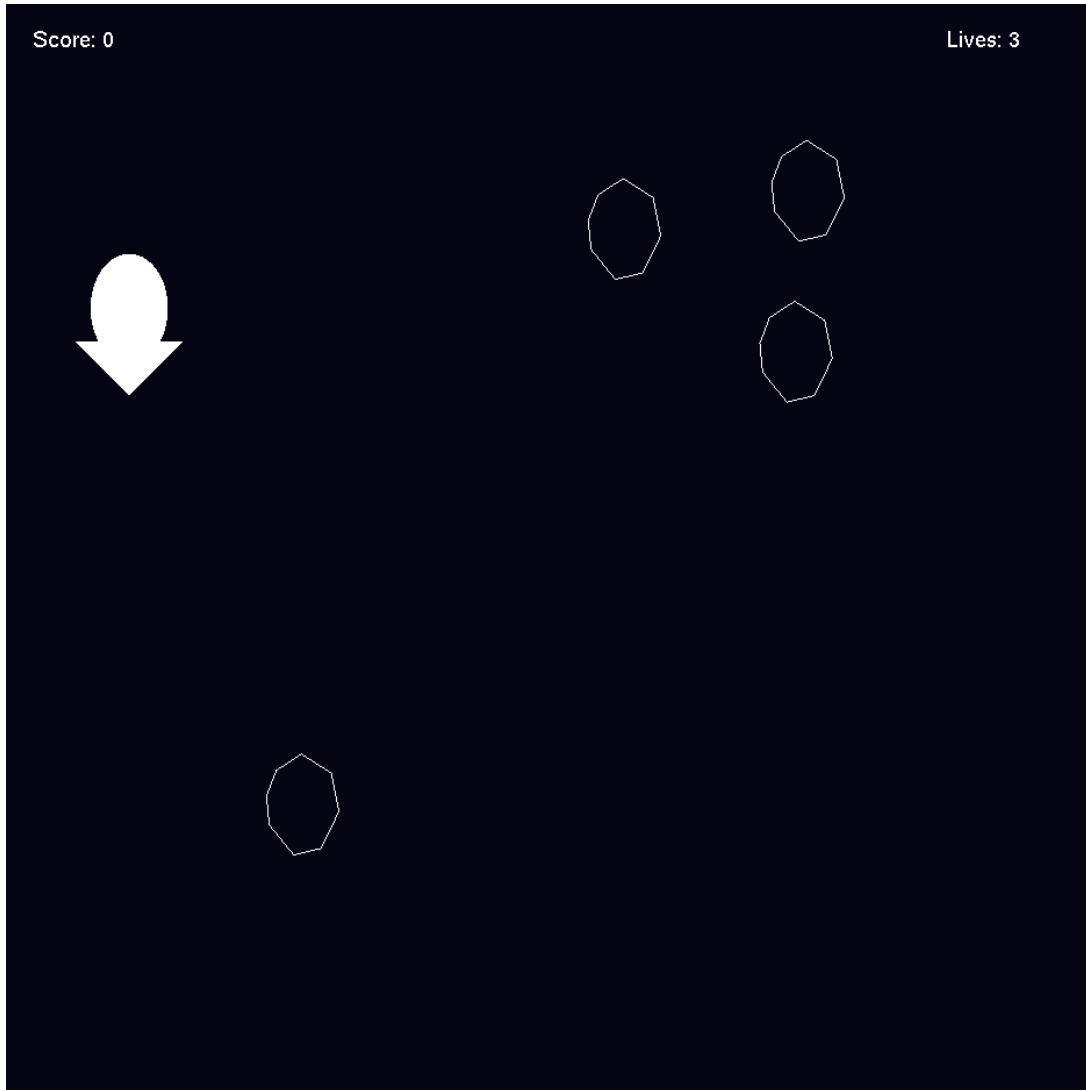
Actual Result

Input/Action

Hold A briefly to move the ship left.

Expected Result

The ship moves left while hazards continue to update.

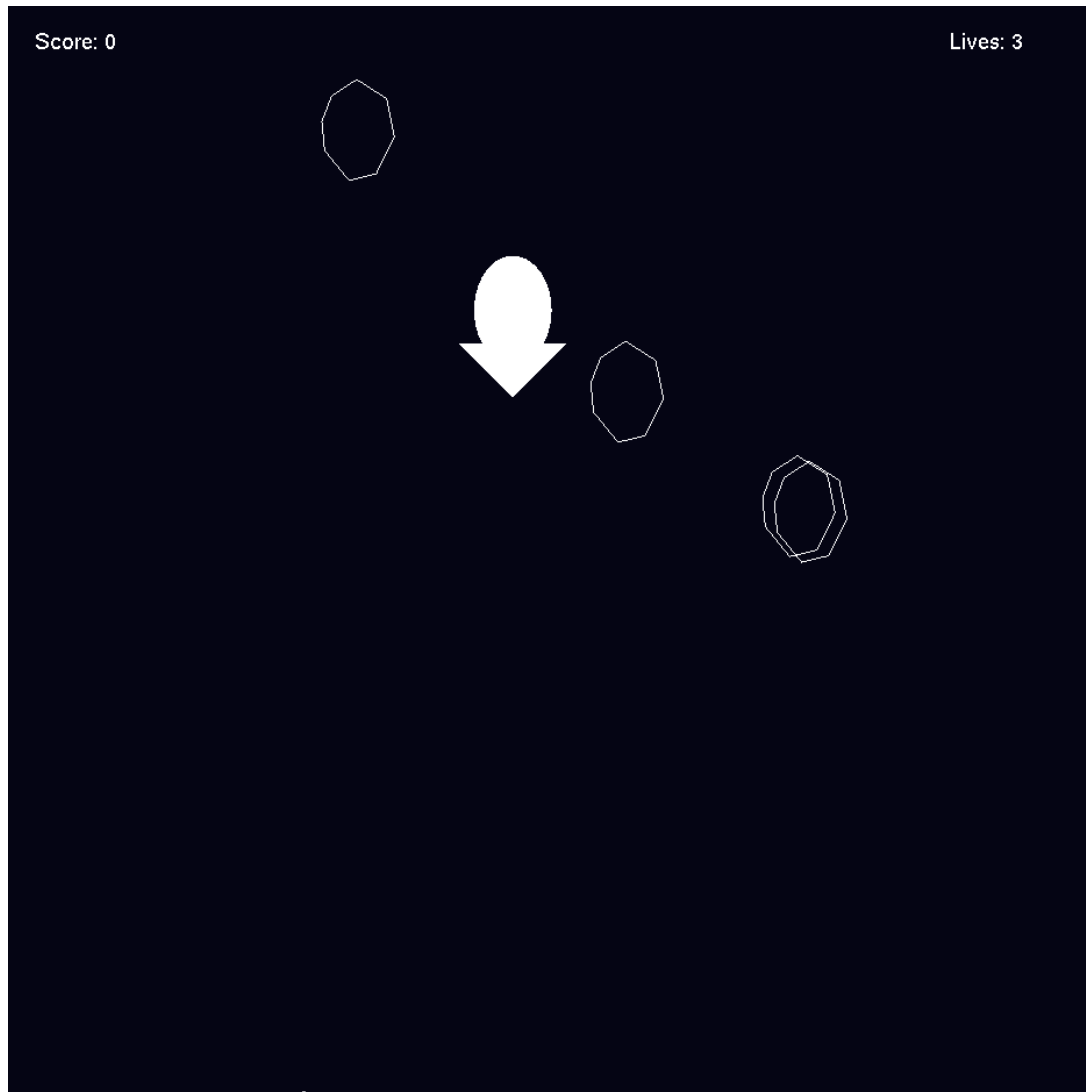
Actual Result

Input/Action

Hold D briefly to move the ship right.

Expected Result

The ship moves right while the HUD remains readable.

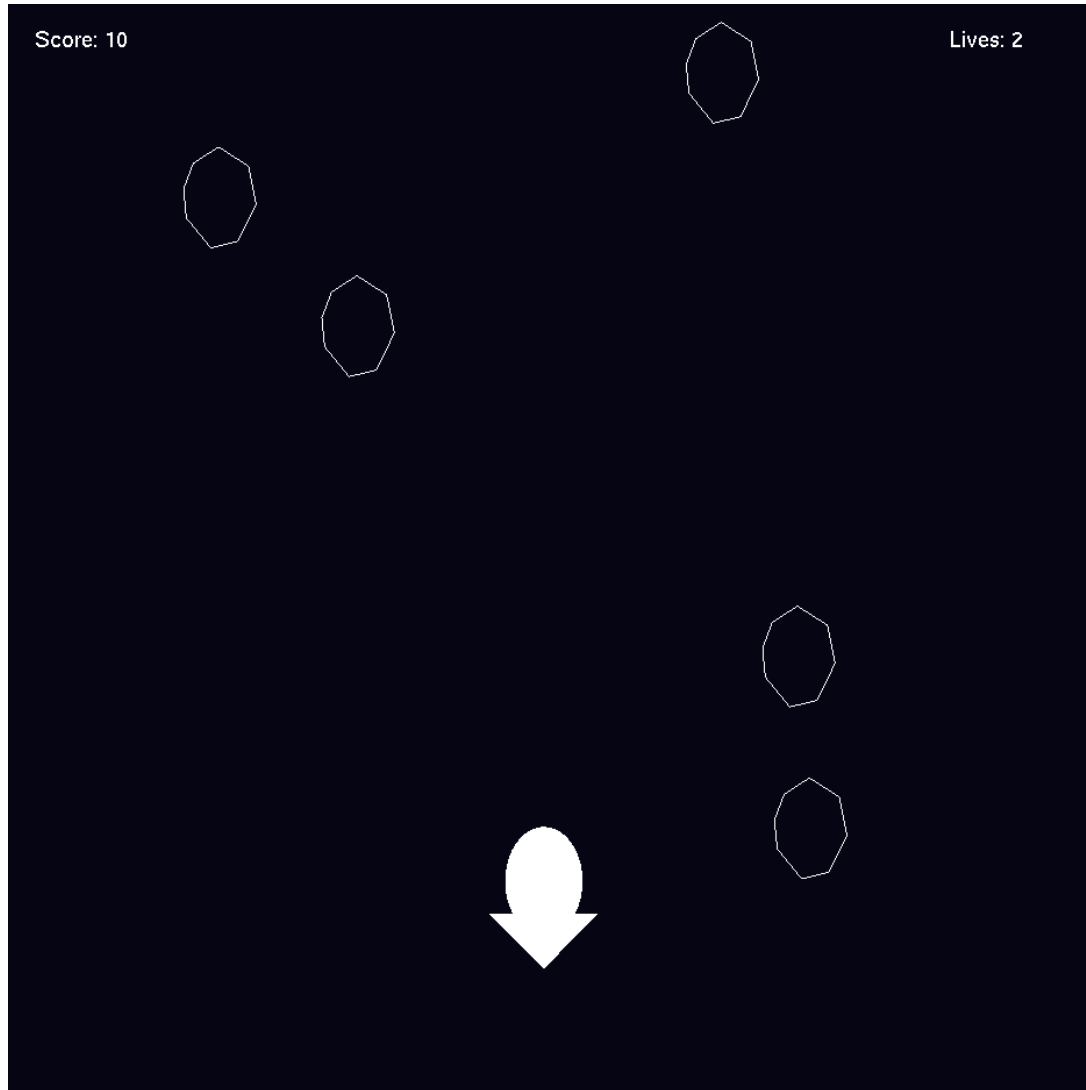
Actual Result

Input/Action

Hold S briefly to move the ship downward.

Expected Result

The ship moves downward while remaining inside the play area.

Actual Result

Machine Problem 2: “Chroma-Shift: The Color-Matching Gate”

Difficulty	Intermediate-Advanced
Concept	Combines the color-state techniques from Programs 1 and 3 with movement, collision logic, scoring, and increasing difficulty.

Objective

Control a central square that changes among Red, Green, and Blue. Incoming barriers carry a color code; the player must match the barrier color to pass safely.

Key Requirements

- **Player Color:** Use R, G, and B to set the player square to red, green, or blue.
- **Moving Barrier:** Create a horizontal or vertical barrier that automatically approaches the player. Each barrier must have one of the three valid colors.
- **Color Matching Logic:** When the barrier reaches the player, compare the player color state with the barrier color state.
- **Scoring:** A correct match increases the score by 1 and resets/spawns the next barrier.
- **Difficulty Scaling:** After a successful match, increase the barrier speed slightly while keeping the game playable.
- **Penalty:** A mismatch must either deduct points or immediately enter a game-over state. State your chosen rule in the on-screen instructions.
- **Dashboard:** Display score, current player color, and concise controls/instructions directly on the window canvas.

Minimum Acceptance Checklist

- ☐ R/G/B must change the player color immediately.
- ☐ The incoming barrier must move automatically.
- ☐ The barrier color must be visually clear before collision.
- ☐ Correct matches and mismatches must produce different outcomes.
- ☐ The speed increase must be tied to successful play.

Recommended Program Structure

- drawPlayer()
- drawBarrier()
- drawDashboard()
- colorsMatch()
- checkGateCollision()
- updateBarrier(int value)
- spawnNextBarrier()
- keyboard(...)
- display()

Reflection Questions

1. What representation did you use for colors: RGB variables, an integer code, or both? Why?

Colors are represented with a `GameColor` enum for game logic and converted into RGB values for drawing.

2. Where should collision be checked: keyboard callback, timer/idle callback, or display()? Explain.

Collision is checked in the timer callback after the barrier moves, so it is evaluated continuously during animation.

3. How did you keep increasing speed from becoming unplayable?

Speed stays playable because each correct match adds a small increase but clamps the barrier speed to a maximum value.

Testing Record

Input/Action

Press R to select red.

Expected Result

The player square and HUD immediately show RED.

Actual Result

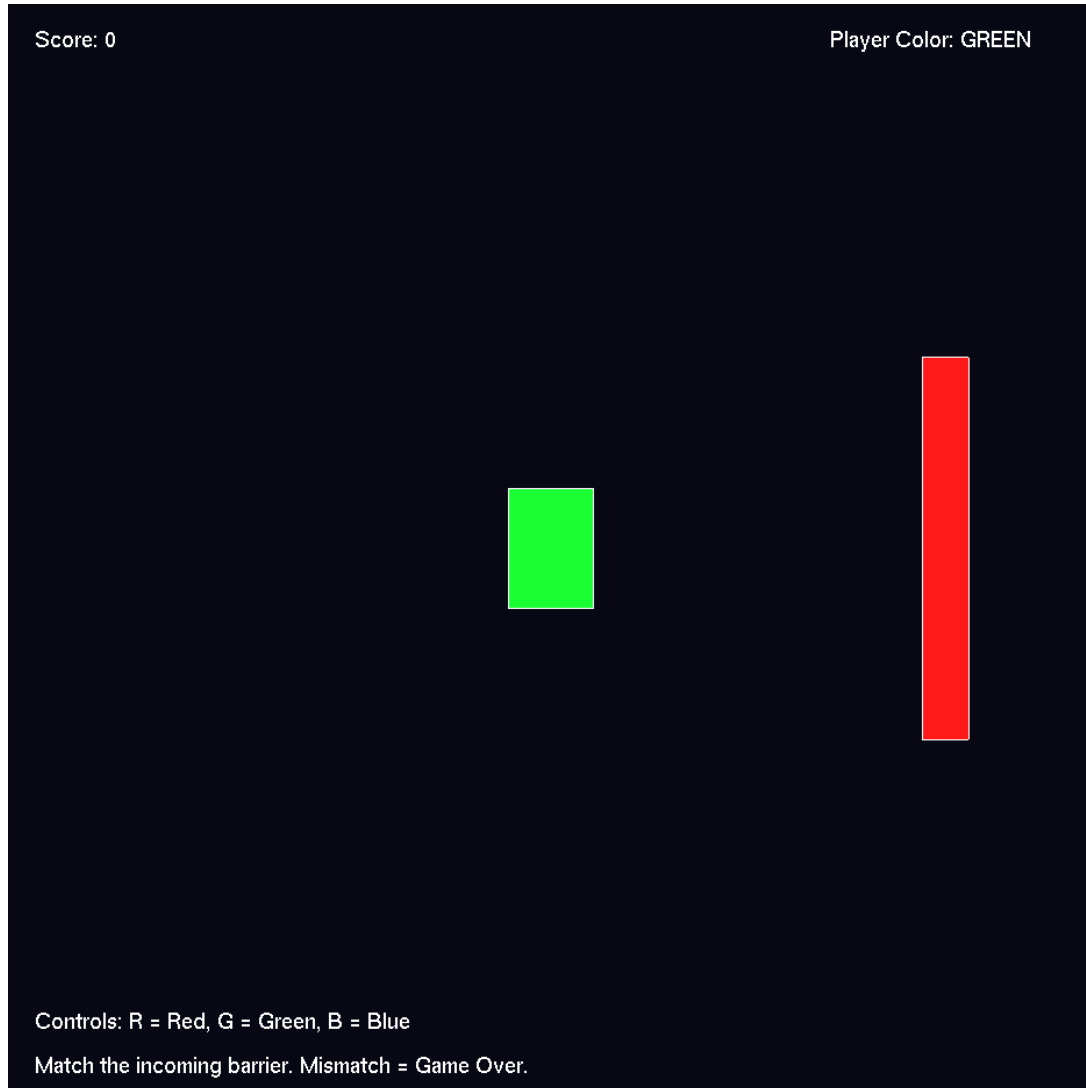


Input/Action

Press G to select green.

Expected Result

The player square and HUD immediately show GREEN.

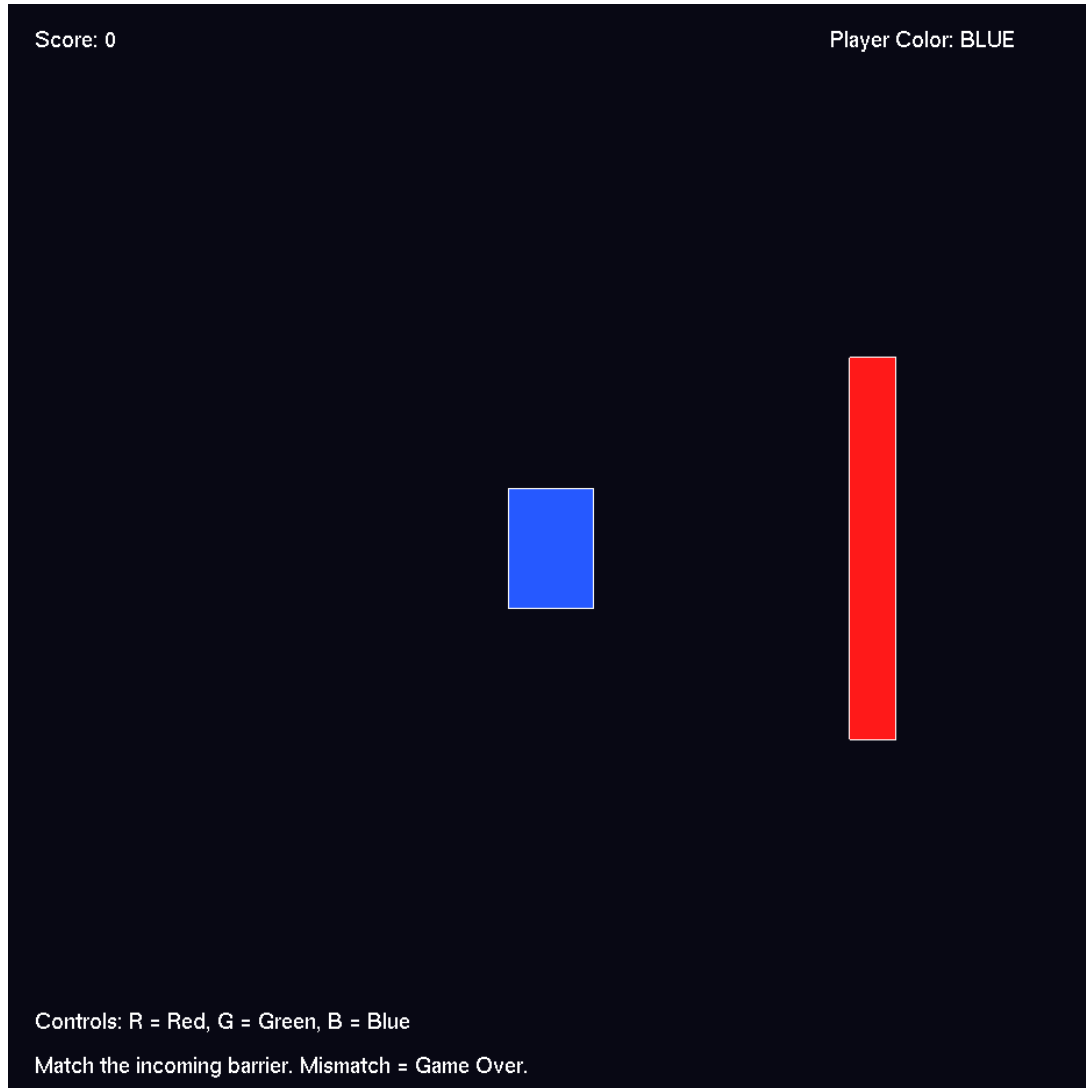
Actual Result

Input/Action

Press B to select blue.

Expected Result

The player square and HUD immediately show BLUE.

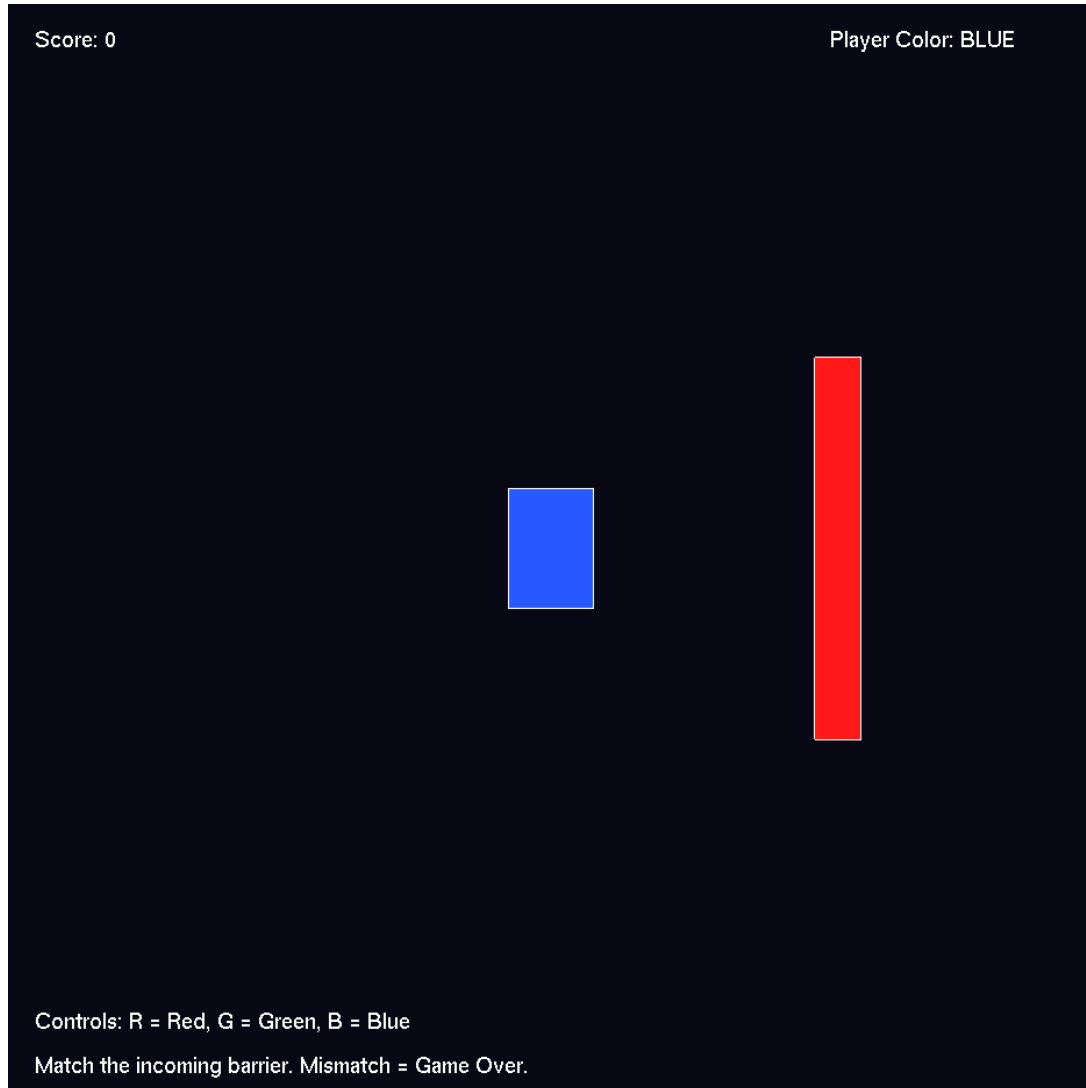
Actual Result

Input/Action

Wait while the incoming barrier approaches.

Expected Result

The barrier continues moving automatically toward the player.

Actual Result

Input/Action

Press R again after the barrier has advanced.

Expected Result

The player returns to RED without stopping the barrier animation.

Actual Result

Machine Problem 3: “Dynamic Fireworks & Shape Burst Studio”

Difficulty	Advanced
Concept	Extends Module 3 timer-driven animation into a small particle-style visual system using only basic OpenGL primitives.

Objective

Create an interactive visual-art generator where keyboard triggers launch animated geometric fireworks that expand and fade over time.

Key Requirements

- **Burst Trigger:** Press SPACE or keys 1, 2, and 3 to create a burst at a randomized position or a stored cursor position.
- **Timer Animation:** Use `glutTimerFunc()` to update active bursts. The timer callback must reschedule itself while animation is needed.
- **Expansion:** Each burst must visibly expand outward by changing particle positions, radius, or scale over successive updates.
- **Fade:** Gradually reduce color intensity or alpha-like brightness over time until the burst expires.
- **Multi-Shape Support:** Key 1: expanding concentric squares; Key 2: diamond lattice; Key 3: starburst triangles, or equivalent clearly distinct patterns.
- **Multiple Active Bursts:** Support more than one burst on screen at the same time. Remove or deactivate expired bursts safely.
- **Control Menu:** Display hotkeys, active particle/burst count, and current rendering mode in a clean sidebar or HUD.

Minimum Acceptance Checklist

- ☐ 1, 2, and 3 must create visibly different patterns.
- ☐ Animation must continue without requiring repeated key presses.
- ☐ At least two bursts should be able to coexist.
- ☐ Expired bursts must stop consuming active slots or being rendered.
- ☐ The HUD must update as the active count changes.

Recommended Program Structure

- `struct/arrays` for burst state
- `spawnBurst(type, x, y)`
- `drawBurst(...)`
- `updateBursts(int value)`
- `drawSidebar()`
- `keyboard(...)`
- `display()`

Reflection Questions

1. Why must `glutTimerFunc()` be called again inside the timer callback for repeated updates?

`glutTimerFunc()` must be scheduled again because it runs only once per call and needs rescheduling for continuous animation.

2. What data must be stored for each active burst?

Each burst stores its position, color, type, radius, expansion speed, brightness, age, lifetime, and rotation.

3. How did you implement fading without relying on advanced shaders?

Fading is implemented by reducing each burst's brightness over its lifetime and multiplying that value into its RGB color.

Testing Record

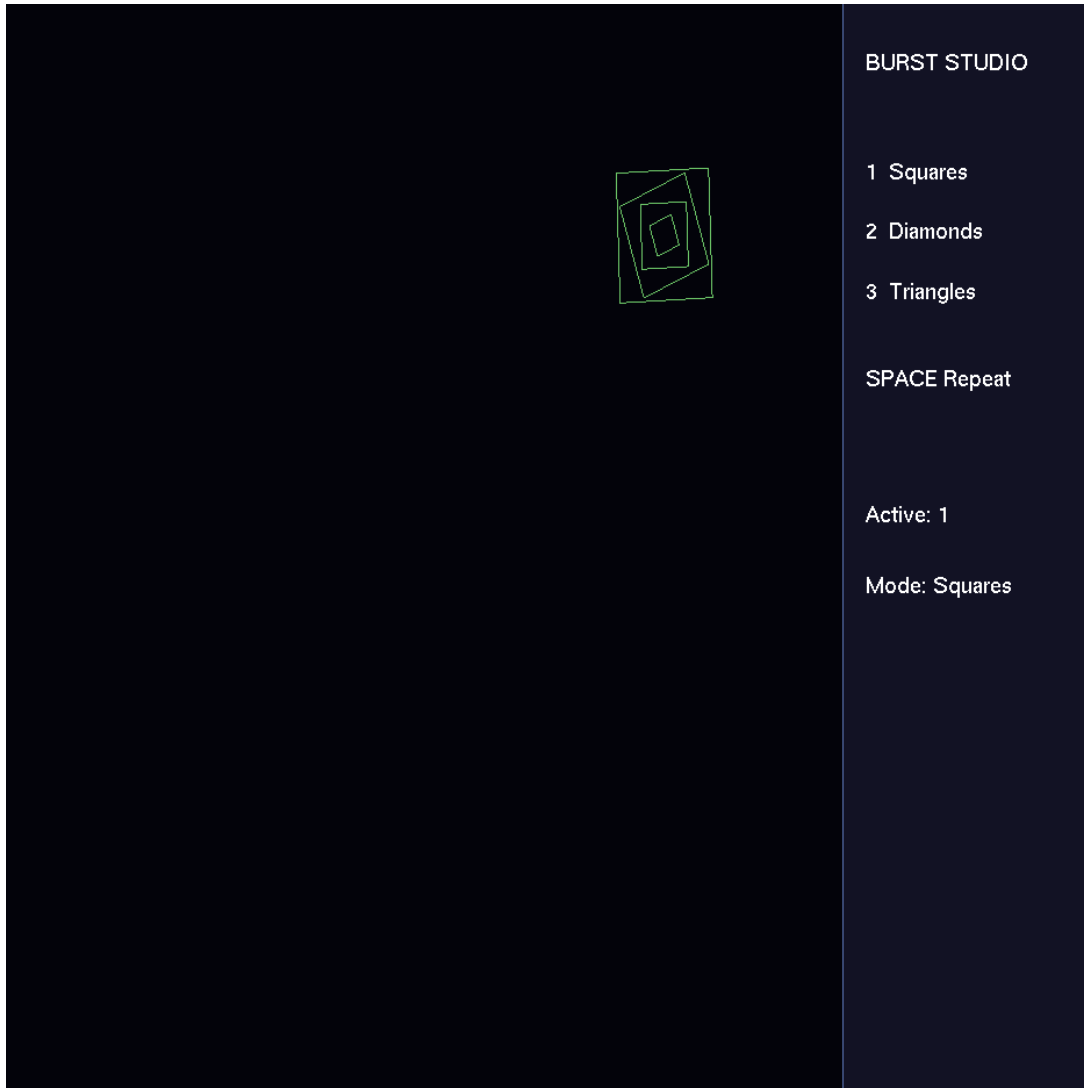
Input/Action

Press 1 to launch a square burst.

Expected Result

An expanding concentric-square pattern appears and the HUD shows Squares mode.

Actual Result

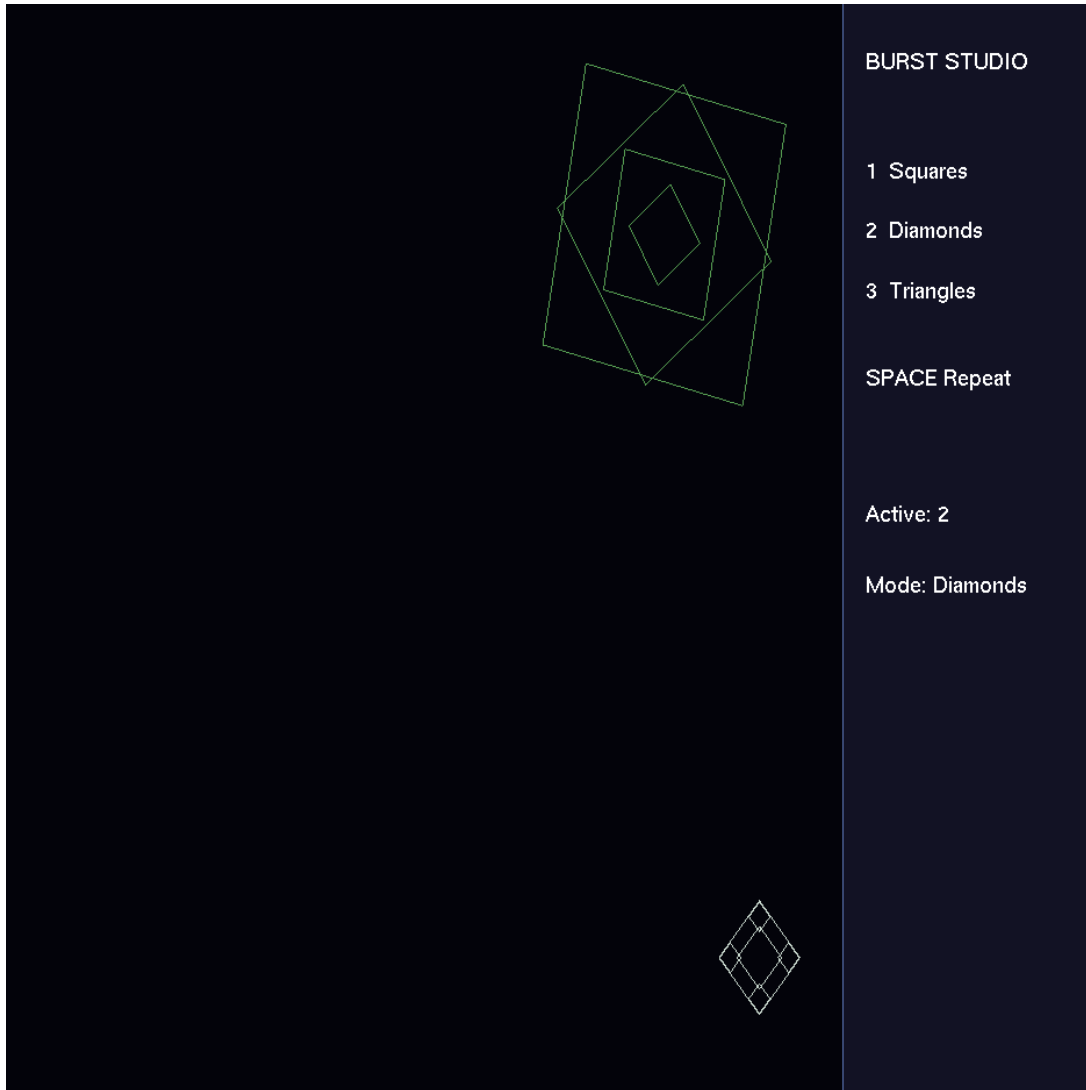


Input/Action

Press 2 while the first burst is active.

Expected Result

A diamond-lattice burst is added and the active count increases.

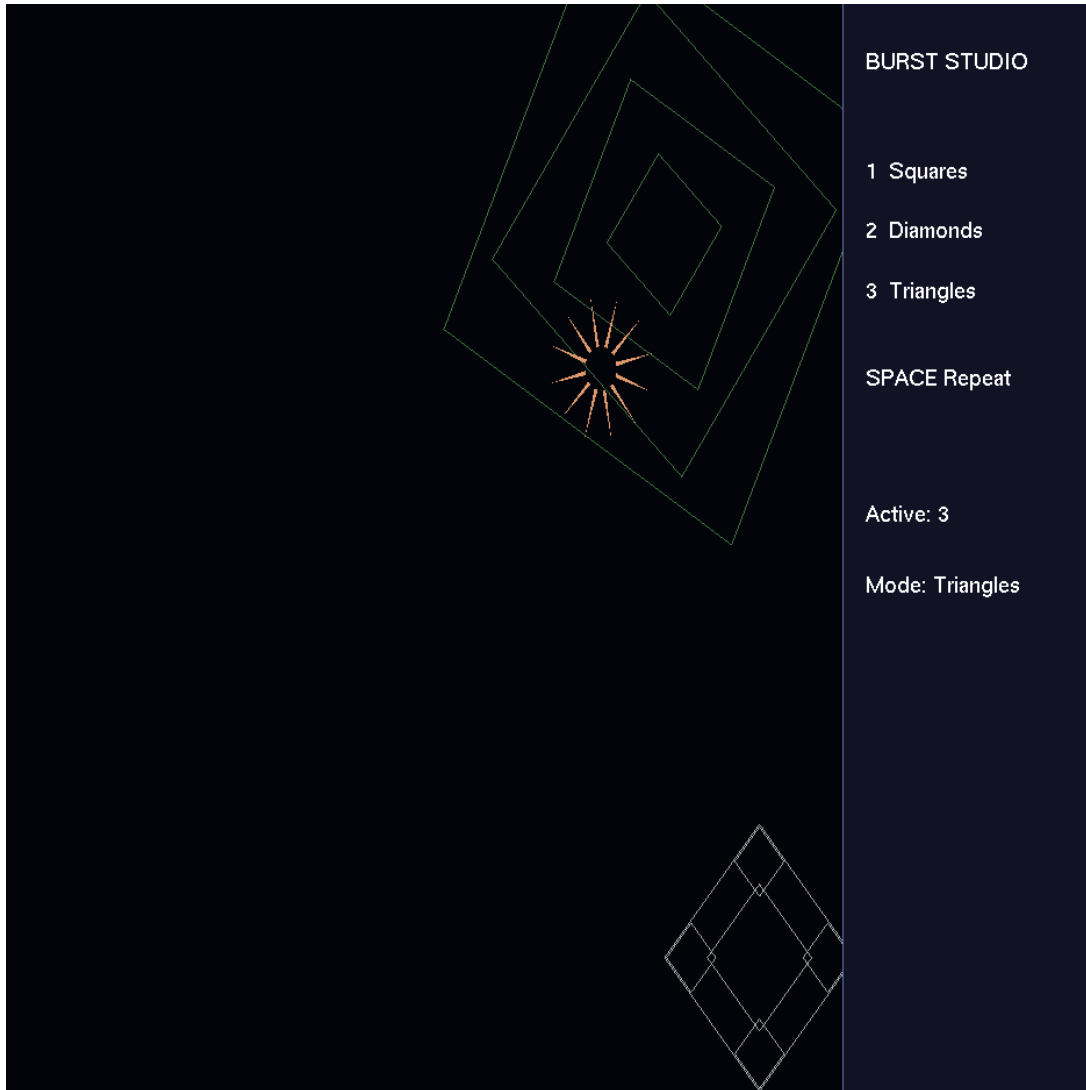
Actual Result

Input/Action

Press 3 while earlier bursts remain active.

Expected Result

A starburst-triangle pattern appears and the active count increases.

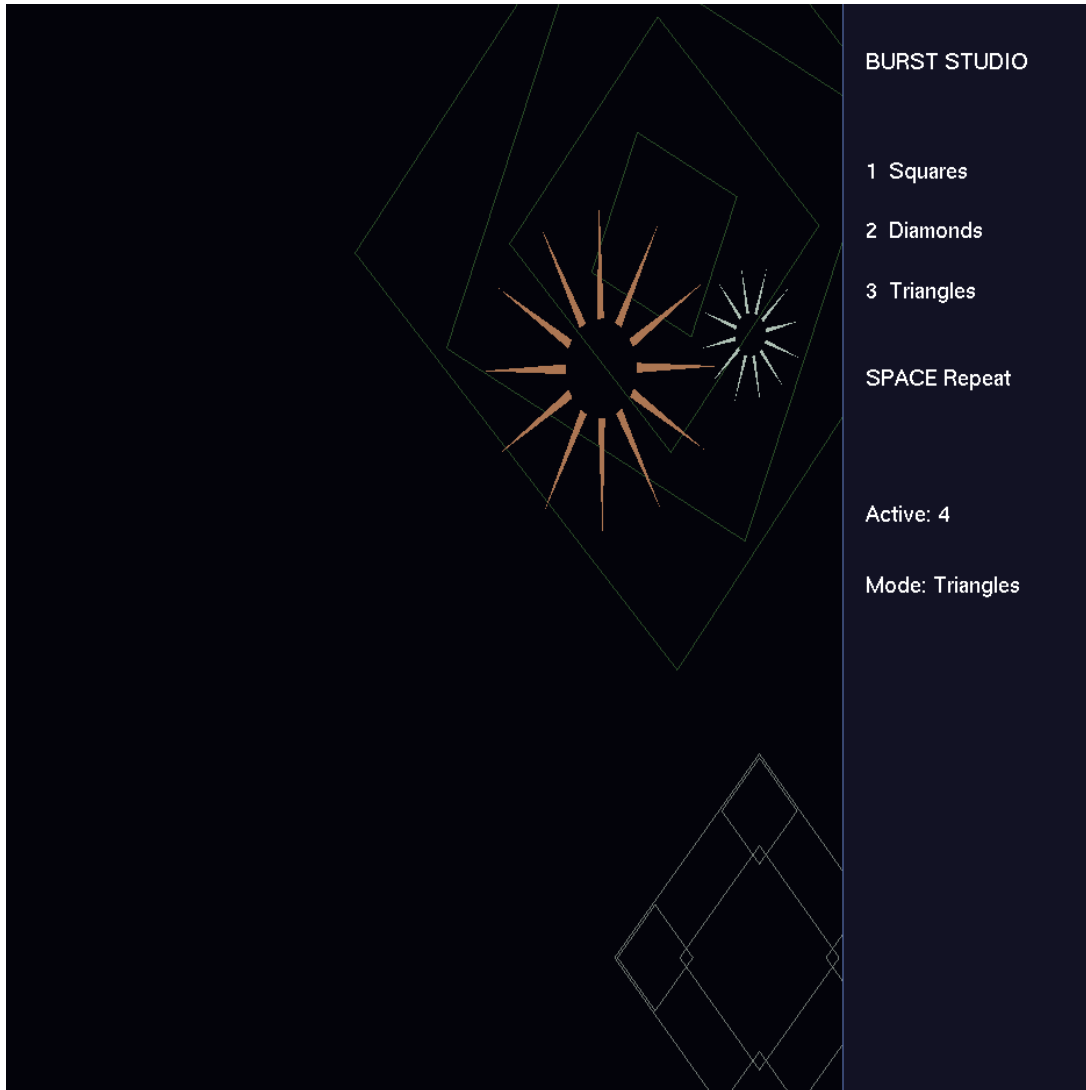
Actual Result

Input/Action

Press Space to repeat the current burst mode.

Expected Result

Another burst is added without removing the current active bursts.

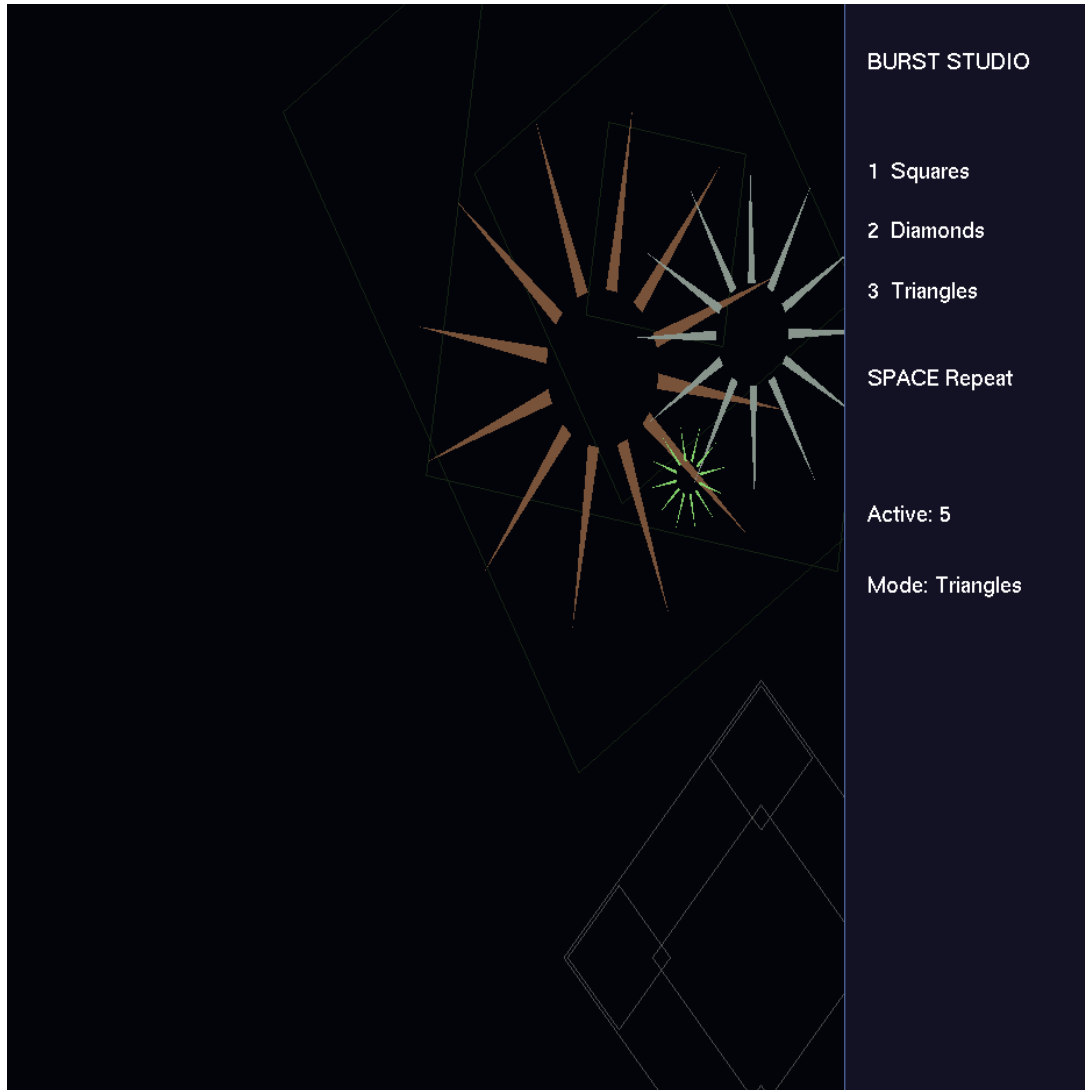
Actual Result

Input/Action

Press Space again while multiple bursts are visible.

Expected Result

Several animated bursts coexist and the HUD reports the updated active count.

Actual Result**SOURCE CODE LINK**

<https://github.com/TGOLiam/CS0045-M3-EXERCISE>

Machine Problem Evaluation Guide

The following common rubric can be applied to each machine problem. Instructors may adjust weights when needed.

Criterion	Weight	Evidence
Functional Requirements	40%	Required controls, states, animation, collision/scoring behavior work correctly.
Graphics & Interaction	20%	Shapes are clear, movement is responsive, HUD/instructions are readable.
Program Logic & Structure	20%	Functions are organized; state is meaningful; repeated logic is minimized.
Robustness & Testing	10%	Boundaries, restart behavior, collision edge cases, and repeated actions are handled.
Code Quality & Reflection	10%	Readable names/comments plus complete reflection and testing record.

Module 3 Concept Map

- Programs 1–3: state changes + keyboard input + redraw.
- Program 4: composite geometry + shared positional offset + bounds.
- Program 5: movement + HUD + collision + score + restart.
- Machine Problem 1: adds continuous hazards, lives, and game states.
- Machine Problem 2: adds state matching, moving gates, and difficulty scaling.
- Machine Problem 3: adds self-rescheduling timers and multiple animated objects.

Reminder The Module 3 reference manual explains that `glutTimerFunc()` schedules a callback once; repeated animation requires the callback to schedule the timer again. It also demonstrates HUD-style bitmap text and continuous animation callbacks.